

ARCHITECTURE

AI Study Companion — Architecture Documentation

AI Study Companion helps students organize their study material and learn more effectively with the help of AI.

1. System Overview

AI Study Companion is a full-stack web application that lets a user organize study material into Spaces → Projects, upload PDF material per project, ask an AI tutor questions grounded in that material (RAG), take AI-generated adaptive quizzes, and track mastery/growth over time. A separate Admin role has a read-only dashboard over usage, learning analytics, and AI-quality metrics.

Stack:

Layer	Technology
Frontend	React 19 + Vite, React Router v7, Axios, Bootstrap 5 (CSS only)
Backend	FastAPI (Python), Uvicorn
Database	MongoDB (via PyMongo)
Vector store	ChromaDB (local persistent client)
Embeddings	<code>sentence-transformers</code> model <code>all-MiniLM-L6-v2</code>
LLM provider	Groq API (<code>groq</code> Python SDK), model <code>openai/gpt-oss-20b</code>
PDF parsing	PyPDF2
Auth	JWT (python-jose, HS256) + bcrypt password hashing (passlib)

2. Repository Layout

```
Backend/
  app/
    main.py          # FastAPI app, CORS, router registration, health check
    config.py         # env-driven configuration
    database.py       # MongoDB client + collection handles
    auth.py           # JWT issuing/decoding, password hashing, auth dependencies
    schemas.py        # Pydantic request/response models
    routers/          # one file per feature area (see §4)
```

```

services/          # business logic + integrations (see §5)
requirements.txt
uploads/           # uploaded PDF files (filesystem)
chroma_store/      # ChromaDB persistent storage (filesystem)

Frontend/
src/
  App.jsx          # routing + route guards
  main.jsx         # app bootstrap, AuthProvider, Bootstrap CSS import
  context/AuthContext.jsx # auth state (localStorage-backed)
  services/api.js   # single Axios instance, JWT header injection
  pages/           # one component per route
  components/      # tab components used inside ProjectDetailPage
  components/admin/ # tab components used inside AdminPage

```

3. Backend Application Wiring

`Backend/app/main.py` creates a single `FastAPI` app titled "AI Study Companion API", adds permissive CORS (`allow_origins=["*"]` , all methods/headers allowed), and mounts 9 routers under distinct prefixes: `/auth` , `/spaces` , `/projects` , `/materials` , `/quiz` , `/tutor` , `/analytics` , `/home` , `/admin` . A single `GET /health` check returns `{"status": "ok"}` .

Configuration (`config.py`) is entirely environment-variable driven via `python-dotenv` :

Variable	Default	Purpose
<code>MONGO_URI</code>	<code>mongodb://localhost:27017</code>	MongoDB connection
<code>DB_NAME</code>	<code>ai_study_companion</code>	Database name
<code>JWT_SECRET</code>	<code>dev-secret-change-me</code>	JWT signing secret (unsafe default)
<code>GROQ_API_KEY</code>	<i>(empty)</i>	Groq API key
<code>UPLOAD_DIR</code>	<code>uploads</code>	Where uploaded PDFs are written
<code>CHROMA_DIR</code>	<code>chroma_store</code>	ChromaDB persistence path

`JWT_ALGORITHM` is fixed to `HS256` , `JWT_EXPIRE_MINUTES` is fixed to `60 * 24 * 7` (7 days), `GROQ_MODEL` is fixed to `openai/gpt-oss-20b` , `EMBEDDING_MODEL` is fixed to `all-MiniLM-L6-v2` .

Database (`database.py`) opens a single `MongoClient` and exposes 13 collections as module-level handles: `users` , `spaces` , `projects` , `materials` , `conversations` , `messages` , `concepts` , `mastery` , `quiz_attempts` , `recommendations` , `activity` , `ai_logs` , `eval_results` . There is no separate migration/index-setup script in the codebase.

4. Authentication & Authorization

Implemented in `Backend/app/auth.py` :

- **Password hashing:** `bcrypt` via `passlib.CryptContext` .
- **Token creation:** `jose.jwt.encode` with payload `{"sub": user_id, "role": role, "exp": <now + 7 days>}` , signed with `JWT_SECRET` using `HS256` . No `iat` / `nbf` / `iss` claims, no refresh-token mechanism, no logout/blacklist, no password-reset flow.
- **Token verification:** `get_current_user` dependency expects an `Authorization: Bearer <token>` header (via `HTTPBearer`), decodes the JWT, and returns `{"user_id", "role"}` to the route.
- **Admin gating:** `require_admin` dependency wraps `get_current_user` and raises `403` if `role != "admin"` . Used by every endpoint in `admin_routes.py` . There is no code path that creates an admin account — `POST /auth/signup` always sets `role="user"` ; admin users must be provisioned directly in MongoDB.

On the frontend, `AuthContext` (`context/AuthContext.jsx`) stores `token` and a `user` object (`{user_id, name, role}`) in `localStorage` , exposes `login` , `signup` , `logout` , and `user` . `services/api.js` is a single `Axios` instance whose request interceptor attaches `Authorization: Bearer <token>` from `localStorage` to every outgoing request. Route protection (`App.jsx`) is client-side only: `ProtectedRoute` redirects to `/login` if no `user` is set; `AdminRoute` redirects to `/admin/login` if no `user` , or to `/home` if `user.role !== "admin"` . There is no token-expiry check on the frontend.

5. Backend Routers & Endpoints

All endpoints except `/auth/*` and `GET /` require a valid JWT; `/admin/*` additionally requires `role == "admin"` .

Prefix	Endpoint	Auth	Purpose
<code>/auth</code>	<code>POST /signup</code>	none	Create user (role fixed to <code>user</code>), returns token
<code>/auth</code>	<code>POST /login</code>	none	Verify credentials, returns token
<code>/spaces</code>	<code>POST /</code>	user	Create a space
<code>/spaces</code>	<code>GET /</code>	user	List own spaces with project counts
<code>/spaces</code>	<code>GET /{space_id}</code>	user	Space detail + its projects
<code>/projects</code>	<code>POST /</code>	user	Create project under an owned space
<code>/projects</code>	<code>GET /{project_id}</code>	user	Project detail, materials, mastery, overall mastery; updates <code>last_accessed</code>
<code>/materials</code>	<code>POST /upload</code>	user	Upload a PDF (multipart), queues background processing
<code>/materials</code>	<code>GET /{material_id}/status</code>	user	Poll processing status (not scoped to owner)
<code>/tutor</code>	<code>POST /ask</code>	user	RAG question answering over a project's materials

Prefix	Endpoint	Auth	Purpose
/tutor	GET /conversations/{project_id}	user	List conversations + messages for a project
/quiz	POST /start	user	Start a 5-question adaptive quiz
/quiz	POST /answer	user	Submit an answer, get next question or completion + recommendation
/quiz	GET /{attempt_id}	user	Get one attempt (answer key stripped)
/quiz	GET /attempts/{project_id}	user	List past attempts for a project
/analytics	GET /{project_id}/overview	user	Per-project usage/performance/AI-activity stats
/analytics	GET /{project_id}/growth	user	Mastery trend per concept
/analytics	GET/POST /{project_id}/recommendation	user	Read latest / generate a new AI recommendation
/analytics	GET /global/overview	user	Cross-project stats for the logged-in user
/home	GET /	user	Dashboard: continue-learning, recent projects, weak concepts, recommendation
/admin	GET /overview	admin	Global platform stats
/admin	GET /users , GET /users/{id}	admin	User listing / detail
/admin	GET /spaces , GET /spaces/{id}	admin	Space listing / detail (cross-user)
/admin	GET /projects , GET /projects/{id}	admin	Project listing / detail (cross-user)
/admin	GET /activity	admin	Paginated, filterable global activity feed
/admin	GET /learning-analytics	admin	Engagement, assessment performance, struggling concepts
/admin	GET /ai-usage	admin	Token/latency/cost aggregation from ai_logs
/admin	GET /evaluation , POST /evaluation/snapshot	admin	Heuristic AI-quality metrics + snapshot history
/admin	GET /system-health	admin	Mongo/Chroma health, stuck materials, recent AI failures

Only 8 Pydantic schemas exist (`schemas.py`): `SignupRequest` , `LoginRequest` , `AuthResponse` , `SpaceCreate` , `ProjectCreate` , `TutorAsk` , `QuizStart` , `QuizAnswer` . Material upload uses raw `Form / UploadFile` parameters; all Admin, Analytics, and Home responses are plain dicts built ad hoc (no declared `response_model`).

6. Material Processing Pipeline

Triggered by `POST /materials/upload`, run as a FastAPI `BackgroundTask` (in-process, not a separate worker/queue):

1. File is saved to `UPLOAD_DIR/{material_id}_{filename}`; a `materials` document is created with `status="queued"`.
2. `pdf_service.extract_text_by_page` extracts text per page (PyPDF2); `pdf_service.chunk_pages` splits it into ~500-word chunks with 50-word overlap, keeping the source page number on each chunk.
3. `vector_service.add_chunks` embeds all chunks with `all-MiniLM-L6-v2` and stores them in a per-project ChromaDB collection (`project_{project_id}`).
4. `concept_service.extract_and_store_concepts` sends the first 5 chunks' text (truncated to 3000 chars) to the Groq LLM to extract 4–6 concept names, upserts them into `concepts`, and seeds a `mastery` record (starting at 30%) for each new concept via `mastery_service`.
5. The material's status is updated to `ready` (with `chunk_count`, `concepts`) or `failed` (with `error`).

Every step is recorded via `activity_service.log_activity` (`MATERIAL_UPLOADED`, `MATERIAL_PROCESSING_COMPLETED` / `MATERIAL_PROCESSING_FAILED`).

7. Tutor (RAG) Flow

`POST /tutor/ask` → `vector_service.search(project_id, question, top_k=4)` queries the project's Chroma collection. If there are no hits, or the closest hit's distance exceeds `RELEVANCE_THRESHOLD = 1.2`, the endpoint returns a fixed refusal string **without calling the LLM**. Otherwise it builds a context string from the top 4 chunks (`[{material_name} - Page {page}]\n{text}`), adds a short "learner note" from `context_service` (built from the user's currently weak concepts, no LLM involved), formats the Tutor prompt, and calls the Groq LLM. The question/answer/sources are persisted to `conversations` / `messages`.

8. Quiz Flow

`POST /quiz/start` picks a concept via a mastery-weighted random selection (`quiz_service.select_next_concept`: weaker concepts are more likely to be picked, never exclusively) and asks the LLM to generate one question (multiple-choice or open-ended). A quiz attempt is 5 questions (`QUIZ_LENGTH = 5`). `POST /quiz/answer` grades multiple-choice answers with plain string comparison (no LLM) and open-ended answers with an LLM grading call; either way, `mastery_service.update_mastery` adjusts the concept's mastery percentage by a difficulty-weighted amount (`easy: 5`, `medium: 10`, `hard: 15`), moving up on a correct answer and down on an incorrect one, clamped to 0–100. On the 5th answer, the attempt is marked completed and `growth_service.generate_recommendation` is called automatically.

9. Analytics, Home & Admin

`growth_service.get_growth_trends` computes a per-concept trend (Improving / Needs Attention / Stable / Not enough data yet) purely from `mastery` history — no LLM. `generate_recommendation` builds a summary of those trends and asks the LLM for 1–2 sentences of guidance (skipped, with a fixed fallback string, if there

are no trends yet). `/home` , `/analytics/*` , and all `/admin/*` endpoints are read-only aggregations over MongoDB collections; the only external calls admin endpoints make are health pings (`db.command("ping")` , `chroma_client.heartbeat()`) — they do not call the LLM or embedder themselves.

10. Frontend Structure

Routing (App.jsx): `/` (Landing), `/login` , `/admin/login` , `/signup` — public; `/home` , `/spaces` , `/spaces/:spaceId` , `/projects/:projectId` — behind `ProtectedRoute` ; `/admin` — behind `AdminRoute` ; unmatched paths redirect to `/login` .

Pages: `LandingPage` (static marketing page), `LoginPage` / `SignupPage` / `AdminLoginPage` (auth forms), `HomePage` (dashboard from `GET /home`), `SpacePage` (list/create spaces), `SpaceDetailPage` (list/create projects in a space), `ProjectDetailPage` (project header + 4 client-side tabs: Materials, Tutor, Quiz, Analytics), `AdminPage` (5 admin tabs: Overview, Users, Spaces & Projects, Learning Analytics, AI Usage).

Feature components (used inside `ProjectDetailPage`): `Materialtab` (upload + poll processing status every 2s), `TutorTab` (chat UI over `/tutor/ask`), `QuizTab` (quiz session UI over `/quiz/start` and `/quiz/answer`), `AnalyticsTab` (per-project stats + recommendation refresh).

Admin components (used inside `AdminPage`): `AdminOverviewTab` , `AdminUsersTab` , `AdminSpacesProjectsTab` , `AdminLearningAnalyticsTab` , `AdminAIUsageTab` . Two additional components exist in `components/admin/` — `AdminSystemHealthTab` and `AdminActivityTab` — that call real, working endpoints (`/admin/system-health` , `/admin/activity`) but are **not imported or wired into** `AdminPage` , so they are not reachable from the current UI.

Styling: Bootstrap 5 CSS only (imported once in `main.jsx`), no Bootstrap JS bundle, no charting library (the Admin learning-analytics bar charts are hand-built from Bootstrap `.progress` divs), no CSS-in-JS/Tailwind.

Build/env: Vite with the default `@vitejs/plugin-react` plugin (no aliases/proxy config). The only frontend environment variable is `VITE_API_BASE_URL` (falls back to `http://127.0.0.1:8000` if unset).